



(12) 发明专利

(10) 授权公告号 CN 115062144 B

(45) 授权公告日 2025. 01. 21

(21) 申请号 202210578307.4

(22) 申请日 2022.05.26

(65) 同一申请的已公布的文献号

申请公布号 CN 115062144 A

(43) 申请公布日 2022.09.16

(73) 专利权人 北京必示科技有限公司

地址 100083 北京市海淀区东升大厦B座
516室

(72) 发明人 曹立 王泓琳 刘大鹏

(74) 专利代理机构 北京中知法苑知识产权代理
有限公司 11226

专利代理师 陈俊由

(51) Int. Cl.

G06F 16/35 (2025.01)

G06F 16/18 (2019.01)

G06F 16/903 (2019.01)

G06F 18/2431 (2023.01)

G06F 18/22 (2023.01)

G06F 18/2433 (2023.01)

G06N 3/0442 (2023.01)

G06N 3/08 (2023.01)

G06N 5/022 (2023.01)

G06N 20/20 (2019.01)

(56) 对比文件

US 2020364104 A1, 2020.11.19

WO 2018122890 A1, 2018.07.05

审查员 靳苗苗

权利要求书1页 说明书8页 附图2页

(54) 发明名称

一种基于知识库和集成学习的日志异常检测方法
与系统

(57) 摘要

本发明公开一种基于知识库和集成学习的日志异常检测方法和系统,其中所述方法包括:获取日志;识别所述日志的类型;基于所述知识库,针对不同日志类型对所述日志进行不同预处理;若所述日志为第一类型日志,则采用预置模板和/或预置规则进行解析;若所述日志为第二类型日志,则基于集成学习进行处理;基于日志类型,对预处理后的日志匹配相应的日志异常检测模型;输出检测结果。本发明使得本发明聚焦于日志异常检测算法在实际生产环境中面临的一些显著挑战,提出基于知识库和集成学习的日志异常检测系统,确保了准确性与可解释性;同时,本发明针对每种日志异常类型,设计并实现合适的异常检测算法,在实际日志数据集中取得了不错的效果。



1. 一种基于知识库和集成学习的日志异常检测方法,包括以下步骤:
 - 获取日志;
 - 识别所述日志的类型;
 - 基于所述知识库,针对不同日志类型对所述日志进行不同预处理;所述知识库至少存储如下信息,常见日志对应的模板、日志解析规则和日志类型对应的异常检测算法;
 - 基于日志类型,对预处理后的日志匹配相应的日志异常检测模型;
 - 输出检测结果;
 - 其中,若所述日志为第一类型日志,则从所述知识库读取预置模板和/或预置规则进行解析;
 - 若所述日志为第二类型日志,则基于集成学习进行处理,其包括:
 - 提取未知类型的日志的关键特征;所述关键特征包括模板数量、模板中实词的数量、模板出现的次数分布和模板之间的相似性;
 - 基于所述关键特征,采用随机森林做分类器,为所述第二类型的日志生成匹配的日志异常检测模型;
 - 其中所述第一类型日志包括通用日志、不适合使用模板进行解析的日志和具有关键变量的日志;
 - 第二类型日志包括未知类型的日志。
2. 如权利要求1所述方法,其中所述日志进行预处理包括将日志的非结构化信息进行处理,以提取结构化信息。
3. 如权利要求1所述方法,其中所述日志异常检测模型包括:模板数量异常检测模型、模板序列异常检测模型、日志变量异常检测模型和关键字异常检测模型。
4. 如权利要求3所述方法,其中所述模板数量异常检测模型采用基于多时间序列的异常检测算法;
 - 所述模板序列异常检测模型采用基于LSTM的模板序列异常检测算法;
 - 所述日志变量异常检测模型采用基于时间序列异常检测的变量取值异常检测算法和基于分布距离的变量分布异常检测算法;
 - 所述关键字异常检测模型采用正则表达式匹配的算法。
5. 如权利要求3所述方法,其中采用多种不同的异常检测模型检测同一日志。
6. 如权利要求1所述方法,其中所述方法输出检测结果包括:日志上下文信息、模板数量的历史规律、模板顺序的工作流图和变量的时序规律。

一种基于知识库和集成学习的日志异常检测方法 with 系统

技术领域

[0001] 本发明属于大数据分析 with 数据挖掘领域,尤其涉及一种基于知识库和集成学习的日志异常检测方法 with 系统。

背景技术

[0002] 日志是软件系统中的一类重要数据源,详细记录了系统的运行状态和 user 行为,日志异常检测,旨在发现系统中的异常行为,可以帮助工程师及时发现系统中的问题并辅助相关的故障诊断。因此,日志异常检测对服务可靠性和故障管理有重要意义。

[0003] 传统监控中,工程师一般基于领域知识,通过手工设置关键字和正则表达式来检测日志异常。但是对着软件服务的规模和复杂性日益增加,通过手工规则的方法有巨大的挑战,包括以下原因。1) 大规模软件服务包含多个模块(如硬件,虚拟机,数据库,网络等),会产生海量、多种多样的日志,大约每天有多达TB级别的日志量。因此手工设置规则耗时耗力;2) 设置规则需要丰富的领域知识,但是有足够专家知识的工程师人数优先,而且不同的工程师在设置规则是有不同的偏好,很难有统一的标准;3) 软件系统会面临频繁的软件变更,导致这些规则需要不断维护和更新。综上,通过手工规则的方法检测日志异常是耗时且易错的。

[0004] 为了解决手工规则的痛点,学术界提出了很多关于日志异常检测的工作。这些工作可以分为五个部分:

[0005] (1) 日志收集。日志收集通常是后续日志分析的第一步。

[0006] (2) 模板解析。日志通常是非结构化的文本,一般需要先通过模板解析把非结构化的原始日志,变成结构化的信息,包含模板与变量。

[0007] (3) 日志分组。由于日志数据是有上下文依赖关系的,我们很难单独根据一条日志判断是否异常,所以需要根据时间窗口对日志进行分组,常见的有滑动窗口等策略。

[0008] (4) 特征提取。现有工作设计了多种方法从窗口中提取特征,基于这些特征进行异常检测,包括模板数量,模板顺序,日志语义特征,变量取值等。

[0009] (5) 异常检测。异常检测的目的是从上一步得到的特征向量中检测出异常,主要包括两大类:1) 传统统计方法,如PCA, 聚类算法等;2) 基于深度学习算法,如近几年流行的基于LSTM的方法。

[0010] 尽管已有工作在公开的日志数据集上的效果得到了一些验证,但是当把这些算法部署到实际生产环境时,我们发现效果并不理想,主要存在以下几点挑战。

[0011] 日志类型多样和日志异常模式多样。软件服务中的每个组件在运行过程中都会产生日志,从硬件层到中间件到应用层。不同种类的日志会表现出不同的异常模式。但是已有工作仅在几个种类单一、异常模式简单的公开数据集上做了效果验证,无法应对实际生产环境中复杂多样的日志类型和异常模式,常见的异常模式包括关键字、模板数量、模板顺序、变量取值、变量分布、时间间隔、日志总量等。

[0012] 可解释性差。已有的工作都是提出一个黑盒算法,直接给出异常检测的判定结果,

但是工程师很难理解算法给出的结果,不明白当前日志为什么是异常、正常的模式应该表现什么样。

[0013] 缺乏领域知识。已有的工作都是端到端的给出异常检测结果,但是面临实际生产中复杂的情况,为了确保检测的准确性和可解释性,融合领域知识是必要的,比如哪些日志变量比较重要需要被检测,这份日志会表现出什么异常模式,适合用什么类型的异常检测算法,这些都需要领域知识。

发明内容

[0014] 针对上述现有技术中存在的缺陷,本发明提出了针对不同日志类型采用合适的异常检测算法。具体来说,针对模板数量异常,本发明提出基于多时间序列的异常检测的模板数量异常检测算法;针对模板序列异常,本发明采用基于LSTM的模板序列异常检测算法;针对日志变量异常,本发明分别提出基于时间序列异常检测的变量取值异常检测和基于分布距离的变量分布异常检测算法。

[0015] 本发明提供一种基于知识库和集成学习的日志异常检测方法,包括以下步骤:

[0016] 获取日志;

[0017] 识别所述日志的类型;

[0018] 基于所述知识库,针对不同日志类型对所述日志进行不同预处理;

[0019] 若所述日志为第一类型日志,则采用预置模板和/或预置规则进行解析;

[0020] 若所述日志为第二类型日志,则基于集成学习进行处理;

[0021] 基于日志类型,对预处理后的日志匹配相应的日志异常检测模型;

[0022] 输出检测结果。

[0023] 其中,所述第一类型日志包括通用日志、不适合使用模板进行解析的日志和具有关键变量的日志;

[0024] 第二类型日志包括未知类型的日志。

[0025] 其中,所述日志进行预处理包括将日志的非结构化信息进行处理,以提取结构化信息。

[0026] 其中,若所述日志为第二类型日志,则基于集成学习进行处理,包括:

[0027] 提取未知类型的日志的关键特征;

[0028] 基于所述关键特征,采用随机森林做分类器,为所述第二类型的日志生成匹配的日志异常检测模型。

[0029] 其中,所述关键特征包括模板数量、模板中实词的数量、模板出现的次数分布和模板之间的相似性。

[0030] 其中,所述日志异常检测模型包括:模板数量异常检测模型、模板序列异常检测模型、日志变量异常检测模型和关键字异常检测模型。

[0031] 其中,所述模板数量异常检测模型采用基于多时间序列的异常检测算法;

[0032] 所述模板序列异常检测模型采用基于LSTM的模板序列异常检测算法;

[0033] 所述日志变量异常检测模型采用基于时间序列异常检测的变量取值异常检测算法和基于分布距离的变量分布异常检测算法;

[0034] 所述关键字异常检测模型采用正则表达式匹配的算法。

- [0035] 其中,采用多种不同的异常检测模型检测同一日志。
- [0036] 其中,所述方法输出检测结果包括:日志上下文信息、模板数量的历史规律、模板顺序的工作流程图和变量的时序规律。
- [0037] 本发明还提出了一种基于知识库和集成学习的日志异常检测系统,其包括:
- [0038] 获取单元,其用于获取日志;
- [0039] 类型识别单元,其用于识别所述日志的类型;
- [0040] 知识库,其包括日志预处理单元和模型匹配单元;
- [0041] 其中,所述日志预处理单元,其用于对所述日志进行预处理,把非结构化的日志文本转换成结构化的信息,用于后续的异常检测;
- [0042] 模型匹配单元,其用于基于集成学习对预处理后的日志匹配相应的日志异常检测模型;
- [0043] 输出单元,其用于输出检测结果。
- [0044] 与现有技术相比,本发明使得本发明聚焦于日志异常检测算法在实际生产环境中面临的一些显著挑战,提出基于知识库和集成学习的日志异常检测系统,确保了准确性与可解释性;同时,本发明针对每种日志异常类型,设计并实现了合适的异常检测算法,在实际日志数据集中取得了不错的效果。

附图说明

- [0045] 通过参考附图阅读下文的详细描述,本公开示例性实施方式的上述以及其他目的、特征和优点将变得易于理解。在附图中,以示例性而非限制性的方式示出了本公开的若干实施方式,并且相同或对应的标号表示相同或对应的部分,其中:
- [0046] 图1是示出根据本发明实施例的一种基于知识库和集成学习的日志异常检测方法流程图;以及
- [0047] 图2是示出根据本发明实施例的一种基于知识库和集成学习的日志异常检测系统的示意图。

具体实施方式

- [0048] 为了使本发明的目的、技术方案和优点更加清楚,下面将结合附图对本发明作进一步地详细描述,显然,所描述的实施例仅仅是本发明一部分实施例,而不是全部的实施例。基于本发明中的实施例,本领域普通技术人员在没有做出创造性劳动前提下所获得的所有其它实施例,都属于本发明保护的范围。
- [0049] 在本发明实施例中使用的术语是仅仅出于描述特定实施例的目的,而非旨在限制本发明。在本发明实施例和所附权利要求书中所使用的单数形式的“一种”、“所述”和“该”也旨在包括多数形式,除非上下文清楚地表示其他含义,“多种”一般包含至少两种。
- [0050] 应当理解,尽管在本发明实施例中可能采用术语第一、第二、第三等来描述……,但这些……不应限于这些术语。这些术语仅用来将……区分开。例如,在不脱离本发明实施例范围的情况下,第一……也可以被称为第二……,类似地,第二……也可以被称为第一……。
- [0051] 应当理解,本文中使用的术语“和/或”仅仅是一种描述关联对象的关联关系,表示

可以存在三种关系,例如,A和/或B,可以表示:单独存在A,同时存在A和B,单独存在B这三种情况。另外,本文中字符“/”,一般表示前后关联对象是一种“或”的关系。

[0052] 取决于语境,如在此所使用的词语“如果”、“若”可以被解释成为“在……时”或“当……时”或“响应于确定”或“响应于检测”。类似地,取决于语境,短语“如果确定”或“如果检测(陈述的条件或事件)”可以被解释成为“当确定时”或“响应于确定”或“当检测(陈述的条件或事件)时”或“响应于检测(陈述的条件或事件)”。

[0053] 还需要说明的是,术语“包括”、“包含”或者其任何其他变体意在涵盖非排他性的包含,从而使得包括一系列要素的商品或者装置不仅包括那些要素,而且还包括没有明确列出的其他要素,或者是还包括为这种商品或者装置所固有的要素。在没有更多限制的情况下,由语句“包括一个……”限定的要素,并不排除在包括所述要素的商品或者装置中还存在另外的相同要素。

[0054] 下面结合附图详细说明本发明的可选实施例。

[0055] 实施例一、

[0056] 如图1所示,本发明公开了一种基于知识库和集成学习的日志异常检测方法,包括以下步骤:

[0057] 获取日志;

[0058] 识别所述日志的类型;

[0059] 基于所述知识库,针对不同日志类型对所述日志进行不同预处理;

[0060] 若所述日志为第一类型日志,则采用预置模板和/或预置规则进行解析;

[0061] 若所述日志为第二类型日志,则基于集成学习进行处理;

[0062] 基于日志类型,对预处理后的日志匹配相应的日志异常检测模型;

[0063] 输出检测结果。

[0064] 实施例二、

[0065] 本发明提出了一种基于知识库和集成学习的日志异常检测方法,包括以下步骤:

[0066] 获取日志;

[0067] 识别所述日志的类型;

[0068] 基于所述知识库,针对不同日志类型对所述日志进行不同预处理;

[0069] 若所述日志为第一类型日志,则采用预置模板和/或预置规则进行解析;

[0070] 若所述日志为第二类型日志,则基于集成学习,使用预置算法进行处理;

[0071] 基于日志类型,对预处理后的日志匹配相应的日志异常检测模型;

[0072] 输出检测结果。

[0073] 其中,接入日志做异常检测时,通常是知道日志类型的。比如现在要接Oracle日志做异常检测,那么oracle这个是先验知识。

[0074] 其中,所述第一类型日志包括通用日志、不适合使用模板进行解析的日志和具有关键变量的日志;

[0075] 第二类型日志包括未知类型的日志。

[0076] 其中,一些关键的变量(如JVM GC日志中的heap使用率)可以通过知识库被提取出来,输入到后面的异常检测过程中。

[0077] 其中,所述日志进行预处理包括将日志的非结构化信息进行处理,以提取结构化

信息。

[0078] 其中,若所述日志为第二类型日志,则基于集成学习进行处理,包括:

[0079] 提取未知类型的日志的关键特征;

[0080] 基于所述关键特征,采用随机森林做分类器,为所述第二类型的日志生成匹配的日志异常检测模型。

[0081] 这里,由于日志中包含丰富的信息,会呈现出多样的日志异常模式,而不同日志适合的异常检测算法也各不相同。对于JVM GC日志,采用模板数量异常检测模型、关键字异常检测模型和变量取值异常检测模型;对于DB2日志,采用模板数量异常检测模型和关键字异常检测模型;对于HDFS日志,采用模板序列异常检测模型。

[0082] 其中,所述关键特征包括模板数量、模板中实词的数量、模板出现的次数分布和模板之间的相似性。

[0083] 也就是说,对于新出现的日志,基于已有的数据,可以构建有监督的分类模型,自动为其分配适合的算法。直观而言,如果未知日志的模式和已经配置在知识库中的日志(有标签数据)相似,那么他们的异常模式和适合的算法也应该是一样的。例如,我们对未知日志提取以下几种特征:模板数量,模板中实词的数量,模板出现的次数分布,模板之间的相似性(用jaccard距离)。基于这些特征,用随机森林做分类器,自动为新日志选择分配合适的算法。当然,如果自动分配的算法检测效果不理想,也可以人工为其选择合适的算法,并将该知识补充到知识库中。

[0084] 其中,所述日志异常检测模型包括:模板数量异常检测模型、模板序列异常检测模型、日志变量异常检测模型和关键字异常检测模型。

[0085] 其中,所述模板数量异常检测模型采用基于多时间序列的异常检测算法;

[0086] 所述模板序列异常检测模型采用基于LSTM的模板序列异常检测算法;

[0087] 所述日志变量异常检测模型采用基于时间序列异常检测的变量取值异常检测算法和基于分布距离的变量分布异常检测算法;

[0088] 所述关键字异常检测模型采用正则表达式匹配的算法。

[0089] 其中,采用多种不同的异常检测模型检测同一日志。

[0090] 其中,所述方法输出检测结果包括:日志上下文信息、模板数量的历史规律、模板顺序的工作流程图和变量的时序规律。

[0091] 综上,采用集成学习,对于日志中的不同数据类型,需要在系统内配置合适的异常检测算法,而不同日志中包含的数据类型不尽相同,因此,需要为每种日志选择其合适的异常检测算法:

[0092] (1) 关键字。日志关键字策略虽然比较简单,但是也是发现故障最直接的形式。通常设置罕见、具体、更致命的关键字(如“OutOfMemory”和“IOException”))对异常检测准确率更有贡献。知识库中可以维护一些常见日志的常见关键字。

[0093] (2) 模板数量。考虑到算法的可解释性,用时间序列刻画模板数量的特征。对历史数据,把每分钟每类模板出现的次数统计称时间序列,模板数量的异常检测即可转换成时间序列的异常检测问题。通常,一份日志有几十到几千条模板不等,为了节约系统资源和减少无效告警数量,我们把全部的模板序列作为一个整体,采用基于LSTM的多时间序列异常检测算法对模板数量进行检测,算法的预测值和真实值的差距作为异常分数。

[0094] (3) 模板顺序。模板的顺序通常刻画了任务的执行顺序,一旦任务执行顺序违反了常规的模式,就认为是异常。本发明中,我们采用LSTM算法来学习正常的模板顺序,同时为了确保可解释性,构建出一个有限状态机清晰地表示任务执行的工作流。详细来说,LSTM算法的输入是 $\{x_1, x_2, \dots, x_w\}$,其中 x_i 表示第 i 条日志对应的模板编号,输出是 x_{w+1} ,即给定前 w 条日志的模板,预测下一条日志是什么。算法会给出当前到达模板的预测结果,一般是一组列表 $\{p_1, p_2, \dots, p_n\}$ 及对应的概率,一旦到达的日志模板不在top-k的预测范围内,就认为是异常。

[0095] (4) 变量取值。变量的取值一般是可以聚合成时间序列的形式,本发明中先用自相关系统判断变量时间序列是否有周期,如果有,则采用holt-winters检测异常;如果没有,则采用3-sigma检测异常。

[0096] (5) 变量分布。有的变量不是数值型的连续变量,而是类别型(如IP地址,返回码),对于这类变量,通常考虑其分布的变化。一般而言,类别的分布是符合一个正常模式的,一旦分布发生变化,可能意味着系统出现了某种问题,如某个IP地址占比增多,可能是这个IP地址一致在尝试访问。本发明中,提出日志变量分布异常检测问题,并采用JS散度(Jensen-Shannon divergence)来刻画变量的分布,一旦当前窗口的分布显著区别于正常时间(距离超过阈值),则认为是异常。

[0097] 实施例三、

[0098] 本发明某一实施例中,知识库中存储至少三类信息。

[0099] (1) 常见日志对应的模板:以hdfs模板举例,这样未来系统要接入新的hdfs日志时,可以直接从知识库中读取出模板,不在需要跑模板提取算法和手工调优。

[0100] (2) 日志解析规则:对于部分特殊的日志,在知识库中存储正则解析规则,比如accesslog可以按照正则来解析成【时间,IP,URL,返回码】

[0101] 111.43.122.194 - - [20/Nov/2021:00:00:00 +0800] "GET /__local/2/A3/01/98E7F968084669CD9F7A4BB8304_FEC00CC4_D5AD5.jpg HTTP/1.1" 200 875221 "https://www.tsinghua.edu.cn/" "Mozilla/5.0 (Linux; U; Android 9; zh-cn; Redmi 8A Build/PKQ1.190319.001) AppleWebKit/537.36 (KHTML, like Gecko) Version/4.0 Chrome/89.0.4389.116 Mobile Safari/537.36 XiaoMi/MiuiBrowser/15.5.12" 解析之后可以得到

[0102] 时间=20/Nov/2021:00:00:00

[0103] IP=111.43.122.194

[0104] URL="https://www.tsinghua.edu.cn/"

[0105] 返回码=200。

[0106] (3) 日志类型对应的异常检测算法。

[0107] 实施例四、

[0108] 如图2所示,本发明还提出了一种基于知识库和集成学习的日志异常检测系统,其包括:

[0109] 获取单元,其用于获取日志;

[0110] 类型识别单元,其用于识别所述日志的类型;

[0111] 知识库,其包括日志预处理单元和模型匹配单元;

[0112] 其中,所述日志预处理单元,其用于对所述日志进行预处理,把非结构化的日志文本转换成结构化的信息,用于后续的异常检测;

[0113] 模型匹配单元,其用于基于集成学习使用预置算法对预处理后的日志匹配相应的日志异常检测模型;

[0114] 输出单元,其用于输出检测结果。

[0115] 实施例五、

[0116] 本公开实施例提供了一种非易失性计算机存储介质,所述计算机存储介质存储有计算机可执行指令,该计算机可执行指令可执行如上实施例所述的方法步骤。

[0117] 需要说明的是,本公开上述的计算机可读介质可以是计算机可读信号介质或者计算机可读存储介质或者是上述两者的任意组合。计算机可读存储介质例如可以是一——但不限于——电、磁、光、电磁、红外线、或半导体的系统、装置或器件,或者任意以上的组合。计算机可读存储介质的更具体的例子可以包括但不限于:具有一个或多个导线的电连接、便携式计算机磁盘、硬盘、随机访问存储器(RAM)、只读存储器(ROM)、可擦式可编程只读存储器(EPROM或闪存)、光纤、便携式紧凑磁盘只读存储器(CD-ROM)、光存储器件、磁存储器件、或者上述的任意合适的组合。在本公开中,计算机可读存储介质可以是任何包含或存储程序的有形介质,该程序可以被指令执行系统、装置或者器件使用或者与其结合使用。而在本公开中,计算机可读信号介质可以包括在基带中或者作为载波一部分传播的数据信号,其中承载了计算机可读的程序代码。这种传播的数据信号可以采用多种形式,包括但不限于电磁信号、光信号或上述的任意合适的组合。计算机可读信号介质还可以是计算机可读存储介质以外的任何计算机可读介质,该计算机可读信号介质可以发送、传播或者传输用于由指令执行系统、装置或者器件使用或者与其结合使用的程序。计算机可读介质上包含的程序代码可以用任何适当的介质传输,包括但不限于:电线、光缆、RF(射频)等等,或者上述的任意合适的组合。

[0118] 上述计算机可读介质可以是上述电子设备中所包含的;也可以是单独存在,而未装配入该电子设备中。

[0119] 可以以一种或多种程序设计语言或其组合来编写用于执行本公开的操作的计算机程序代码,上述程序设计语言包括面向对象的程序设计语言——诸如Java、Smalltalk、C++,还包括常规的过程式程序设计语言——诸如“C”语言或类似的设计语言。程序代码可以完全地在用户计算机上执行、部分地在用户计算机上执行、作为一个独立的软件包执行、部分在用户计算机上部分在远程计算机上执行、或者完全在远程计算机或服务器上执行。在涉及远程计算机的情形中,远程计算机可以通过任意种类的网络——包括局域网(LAN)或广域网(WAN)——连接到用户计算机,或者,可以连接到外部计算机(例如利用因特网服务提供商来通过因特网连接)。

[0120] 附图中的流程图和框图,图示了按照本公开各种实施例的系统、方法和计算机程序产品的可能实现的体系架构、功能和操作。在这点上,流程图或框图中的每个方框可以代表一个模块、程序段、或代码的一部分,该模块、程序段、或代码的一部分包含一个或多个用于实现规定的逻辑功能的可执行指令。也应当注意,在有些作为替换的实现中,方框中所标注的功能也可以以不同于附图中所标注的顺序发生。例如,两个接连地表示的方框实际上可以基本并行地执行,它们有时也可以按相反的顺序执行,这依所涉及的功能而定。也要注

意的是,框图和/或流程图中的每个方框、以及框图和/或流程图中的方框的组合,可以用执行规定的功能或操作的专用的基于硬件的系统来实现,或者可以用专用硬件与计算机指令的组合来实现。

[0121] 描述于本公开实施例中所涉及到的单元可以通过软件的方式实现,也可以通过硬件的方式来实现。其中,单元的名称在某种情况下并不构成对该单元本身的限定。

[0122] 以上介绍了本发明的较佳实施方式,旨在使得本发明的精神更加清楚和便于理解,并不是为了限制本发明,凡在本发明的精神和原则之内,所做的修改、替换、改进,均应包含在本发明所附的权利要求概括的保护范围之内。

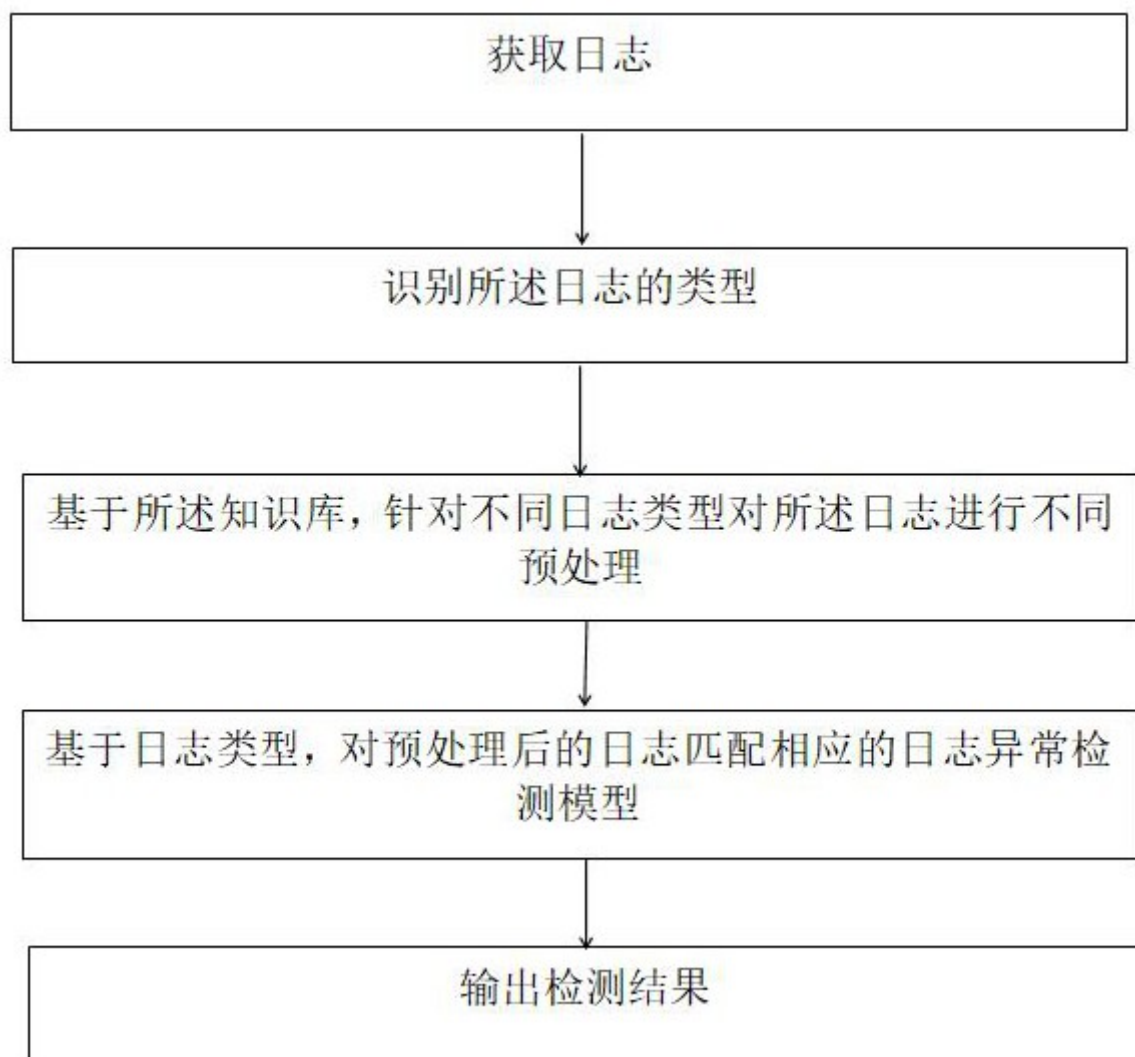


图1



图2